

Rank	Prediction	Why?	Exact Confirmation Test	Observation that falsifies/confirms
1	Concurrent idempotency check-then-insert race condition creates duplicate transactions.	PISMO-4412 explicitly mentions duplicate debit under retry. I've mentioned it as the primary root-cause hypothesis in T1 document. The contract also is silent on idempotency behavior.	TXN-18 Steps: 1. Create an isolated account 2. Generate idempotency key 3. Send N concurrent POST /transactions requests with the same key and body 4. Query a transaction-state/count oracle.	If broken: transaction_record_count > 1 for the account and idempotency key. If correct: count == 1. "POST /transactions" response equality alone is not sufficient, need a transaction-state/count oracle.
2	Idempotency key is not bound to the request payload.	I've mentioned incorrect idempotency key-fingerprint handling as a second root-cause hypothesis in T1 document. Also, same-key/different-body behavior is undefined in the contract.	TXN-17 Steps: 1. POST transaction A with idempotency key K 2. POST transaction B with the same idempotency key K but with different amount/account/operation_type_id. 3. Validate response and transaction state.	If broken: second request is accepted as a new transaction, modifies state. If correct: conflict and no second financial mutation.
3	Operation type is mapped to the wrong financial sign or transaction type	amount sign or transaction type not defined for each operation ID. The contract gives one negative example but does not define the complete mapping. Backend engineer says "Purchases and withdrawals go negative, credit vouchers go positive." which is not contractual.	TXN-1, TXN-2, TXN-3, TXN-4 Steps: 1. Create an account. 2. Submit +ve amount for each operation_type_id. 3. Validate response and storage state with the documented/hearsay mapping.	If broken: any operation produces the wrong sign/type. Example, a voucher becomes debit/negative or a purchase becomes credit/positive. If correct: each operation ID maps to its expected direction/type.
4	Losses due to amount precision rounding issues	Precision/unit rules are missing in the contract. Backend engineer says "caller sends rupees and storage uses the smallest unit", but is not contractual.	TXN-5, TXN-6, TXN-10, TXN-11 Steps: 1. Submit integer, normal decimal, 10.123456 and 0.01 values for amount. 2. Validate returned amount and smallest unit state (if test oracle available).	If broken: returned amount differs from caller amount beyond documented rounding. If correct: amount consistent with the agreed currency precision.
5	Transaction creation does not validate account_id	The system may not validate account_id correctly, allowing a transaction to be created for an account that does not exist. The contract includes an "account not found" error, but it does not clearly define whether this should return 400 or 422.	TXN-13 Steps: 1. Use a valid but non-existent account_id and a valid transaction payload. 2. Validate status and transaction state.	If broken: POST /transactions returns success or creates a transaction for the non-existent account. If correct: deterministic 4xx and no transaction state created.
6	Transaction and audit record can diverge under partial failure	Though the hearsay says both writes are in one DB transaction, it needs to be verified.	Integration / Fault-Injection Test 1. Force audit persistence to fail after the business write is attempted. 2. Inspect both transaction and audit stores. 3. Verify vice-versa as well.	If broken: transaction exists without matching audit row, or audit row exists without business transaction. If correct: neither transaction nor audit row exists.
7	Concurrent/duplicate account creation can bypass document_number uniqueness	The contract does not define how duplicate document_number values should be handled. Concurrent requests could create duplicate accounts if the database/logic does not enforce uniqueness.	ACC-7 1. Submit two creates with the same document_number. 2. Repeat concurrently. 3. Validate response and persisted state.	If broken: multiple accounts are created for one document_number. If correct: one account is created and the duplicate attempt receives a conflicting error.